

迷路で学ぶ 探索アルゴリズム入門

幅優先探索 (BFS)

完成デモ

→ `demo/bfs_visualizer.html` をブラウザで開く

- スタートからゴールまで **何手** かかるか
- ゴールに **到達** できるか
- 最短経路は **どこ** を通るか

今日のゴール

1. スタートからゴールへ **到達できるか** 調べる
2. ゴールまでの **最短距離** を求める
3. ゴールまでの **最短経路** を復元する

今日の流れ

	内容
1	スタックとキューを理解する
2	ミニ演習（動作確認）
3	BFS をアニメーションで追う
4	BFS を実装する（メイン）
5	最短距離・到達不能を確認
6	最短経路の復元

迷路の形式

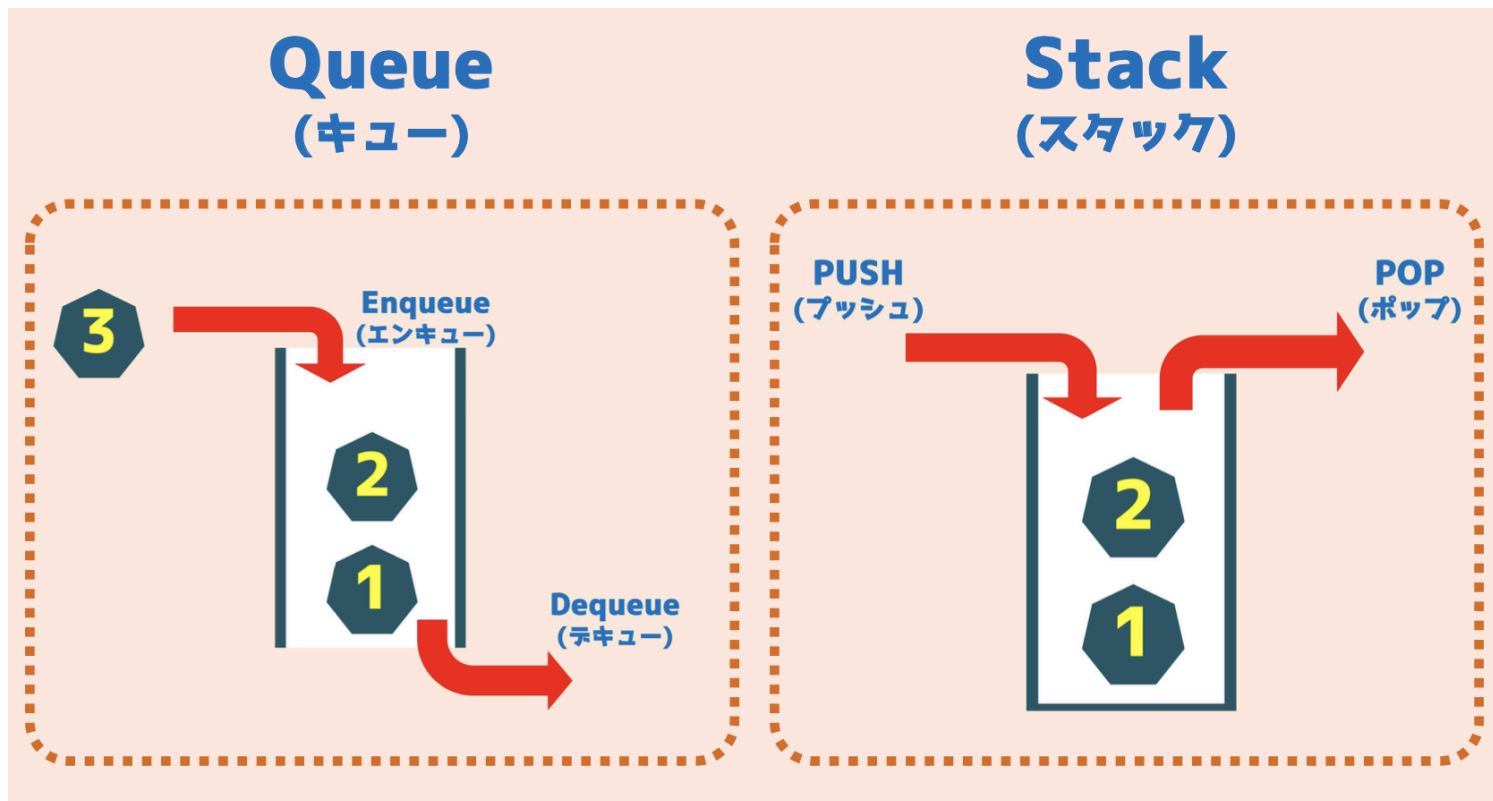
```
5 8      ← 高さ H と幅 W
0 0      ← スタート (row, col)
4 7      ← ゴール   (row, col)
```

```
.....
.####.
...#.
.##....
.....
```

-  : 通れるマス  : 壁
- 座標は **0始まり**
- 上下左右の 4 方向に移動、1 移動 = 1 手

スタックとキュー

スタックとキューのイメージ



スタック

最後に入れたものから取り出す (LIFO)

```
push 1 → push 2 → push 3
```

```
[ 1, 2, 3 ]
```

```
      ↑ pop (ここから取り出す)
```

```
取り出す順: 3 → 2 → 1
```

例え: **本の積み重ね**

スタックコード

Python

```
st = []
st.append(1)
st.append(2)
st.append(3)

while st:
    print(st.pop())    # 3, 2, 1
```

C++

```
stack<int> st;
st.push(1); st.push(2); st.push(3);

while (!st.empty()) {
    cout << st.top() << '\n';    // 3, 2, 1
    st.pop();
}
```

キュー

最初に入れたものから取り出す (FIFO)

```
push 1 → push 2 → push 3
```

```
[ 1, 2, 3 ]
```

```
  ↑ pop (ここから取り出す)
```

```
取り出す順: 1 → 2 → 3
```

例え: レジの行列

キューーコード

Python

```
from collections import deque
que = deque()
que.append(1); que.append(2); que.append(3)

while que:
    print(que.popleft())    # 1, 2, 3
```

C++

```
deque<int> que;
que.push_back(1); que.push_back(2); que.push_back(3);

while (!que.empty()) {
    cout << que.front() << '\n';    // 1, 2, 3
    que.pop_front();
}
```

ミニ演習

`python/2_stack_queue_exercise.py` または `cpp/2_stack_queue_exercise.cpp`

10, 20, 30, 40 を順番に追加して...

問題 1: スタックで `40 → 30 → 20 → 10` の順に表示する

問題 2: キューで `10 → 20 → 30 → 40` の順に表示する

演習の答え

問題 1 (スタック)

```
st = []  
for n in [10, 20, 30, 40]:  
    st.append(n)  
while st:  
    print(st.pop())
```

問題 2 (キュー)

```
que = deque()  
for n in [10, 20, 30, 40]:  
    que.append(n)  
while que:  
    print(que.popleft())
```

幅優先探索 (BFS)

BFS のイメージ

スタート $(0, 0)$ から 近い順 に調べる

```
  0 1 2 3
0 0 1 2 3
1 1 # 3 4
2 2 3 4 G
```

- 距離 $0 \rightarrow 1 \rightarrow 2 \rightarrow \dots$ と波が広がる
- キューに入れた順 = 近い順 に処理される

アニメーションで確認

→ `demo/bfs_visualizer.html` を **1手ずつ** 進める (← → キー)

確認するポイント:

1. キューにスタートを入れる
2. キューの先頭を取り出す (「今」のマス)
3. 上下左右を確認する
4. 行ける場所を距離付きでキューに追加 (「待」のマス)
5. 一度見たマス (数字が入っているマス) は **もう見ない**

なぜキューで最短距離が求まるか

キューは先に入れたものから出る

↓

近いマスから順番に処理される

↓

あるマスに最初に到達したとき = 最短距離

↓

一度訪れたマスは二度と調べない

BFS の手順（疑似コード）

スタートをキューに入れる

while キューが空ではない:
 現在地をキューから取り出す

 上下左右について:
 次の場所を求める

 迷路の外ならスキップ
 壁ならスキップ
 すでに訪問済みならスキップ

 距離を記録する
 キューに追加する

BFS を実装しよう

上下左右への移動

方向	row の変化	col の変化
上	-1	0
下	+1	0
左	0	-1
右	0	+1

```
dr = [-1, 1, 0, 0]
dc = [0, 0, -1, 1]
```

```
next_row = row + dr[i]    # i 番目の方向へ進んだ先
next_col = col + dc[i]
```

迷路の外に出ていないか確認

移動先が迷路の外になることがある。壁チェックより先に行う

Python

```
if not (0 <= next_row < H and 0 <= next_col < W):  
    continue
```

C++

```
if (next_row < 0 || next_row >= H ||  
    next_col < 0 || next_col >= W) {  
    continue;  
}
```

キューに座標を入れる

Python

```
que.append((row, col))      # 追加  
row, col = que.popleft()   # 先頭を取り出す
```

C++

```
deque<pair<int, int>> que;  
  
que.push_back({row, col});    // 追加  
  
auto [row, col] = que.front(); // 先頭を見る  
que.pop_front();             // 先頭を削除
```

TODO を埋めよう

python/3_bfs_template.py / cpp/3_bfs_template.cpp

TODO	内容
1	キューから現在地を取り出す
2	次の座標を求める
3	迷路の外ならスキップ
4	壁ならスキップ
5	訪問済みならスキップ
6	距離を記録する
7	キューに追加する

正解の出力 → **最短距離: 11**

最短距離を確認する

BFS が終わったら `dist[goal_row][goal_col]` を確認する

```
answer = dist[goal_row][goal_col]

if answer == -1:
    print("ゴールには到達できません")
else:
    print("最短距離:", answer)
```

- `-1` のまま → 到達できない
- `0` 以上 → その値が最短距離

到達できない迷路

maze05_unreachable.txt

```
.....  
#####  
  
.....  
#####  
  
.....
```

スタート (0, 0) → ゴール (4, 6)

キューが空になれば BFS は必ず止まる
到達不能用の特別な処理は要らない

経路を復元する（デモ）

→ `answers/3_bfs_path_answer.*` を実行して見せる

考え方: 各マスに「どこから来たか」を記録する

```
prev_row[next_row][next_col] = row  
prev_col[next_row][next_col] = col
```

ゴールから逆にたどると...

```
S.....  
*####...  
*...#...  
*##.....  
*****G
```

別の迷路で試す

`python/4_bfs_from_file.py` / `cpp/4_bfs_from_file.cpp`

`maze_file` を書き換えると別の迷路で試せる

ファイル	特徴
<code>maze04.txt</code>	少し複雑
<code>maze05_unreachable.txt</code>	到達不能
<code>maze11_detour.txt</code>	見た目 14 手、実際 30 手

DFS と BFS の違い

	BFS（幅優先）	DFS（深さ優先）
データ構造	キュー	スタック / 再帰
探索の順番	近い順	深い方向へ進む
最短距離	求まる	求まらない
到達確認	できる	できる

最短距離が欲しいときは **BFS**

まとめ

今日の重要ポイント

- BFS では **キュー** を使う
- **近い場所から順番**に探索する
- 同じ場所を **何度も調べない**（訪問済み管理）

BFS の応用

同じ考え方がいろんな場面で使える

- ゲームのマップ探索
- SNS のつながり（O度の知り合い）
- ネットワークの経路探索
- パズルの最短手数

おつかれさまでした！